



Introduction to Machine Learning and Classification Problems

Marco Laudato – KTH Department of Engineering Mechanics
Courses and Seminar Series for Free Credits Activities, Naples (IT), 07th December 2024

Overview of the lectures

We will cover the following curriculum :

(Github repo: <https://github.com/MarcoLaud/VP2024/tree/main/UniNaArch>):

- Introduction and motivation to scientific machine learning
 1. Neural Networks and architectures
 2. Classification problems
- Implementation of a classification problem in Grasshopper (w/ Giovanni)

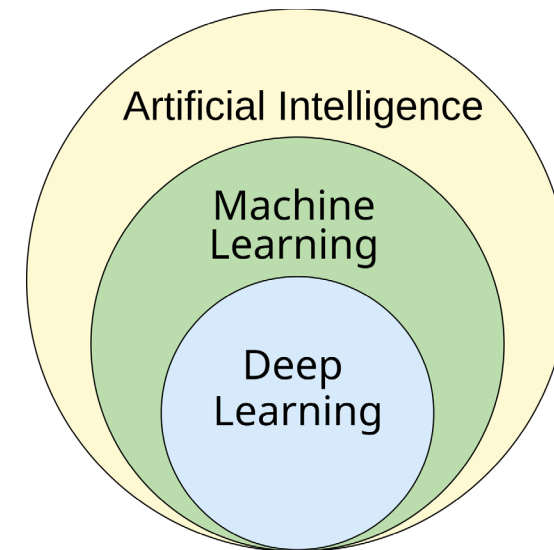
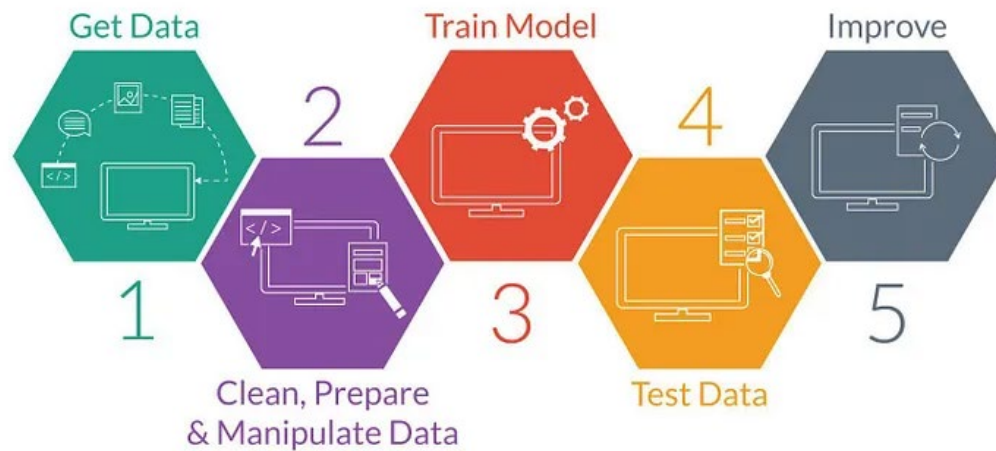
Learning goals:

- Understand basics of neural networks and their application
- Understand basics of classification problems
- Train your first neural network in Grasshopper



What is Machine Learning?

- Machine Learning concerns training computers to learn from data and make predictions.
- The term was coined in **1959** (!) by Arthur Samuel (IBM).
- Applications: healthcare (e.g., diagnosis), finance (e.g., algorithmic trading), transportation (e.g., autonomous vehicles), daily life (e.g., LLMs).

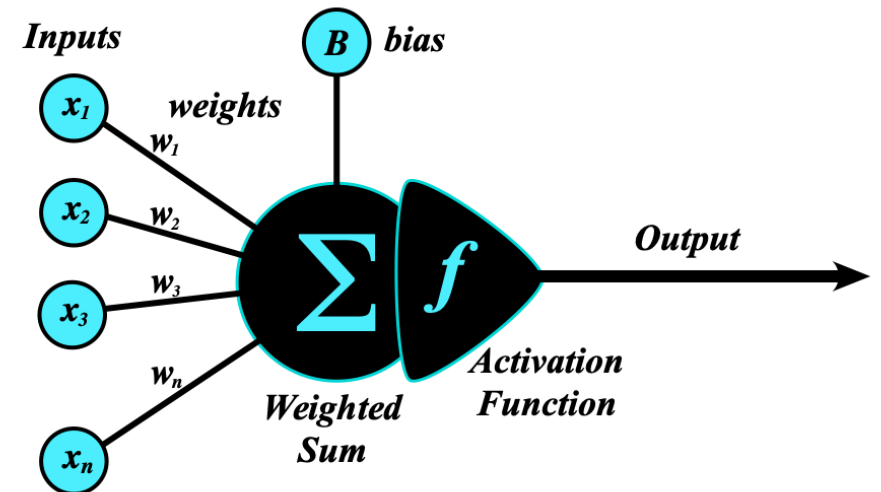
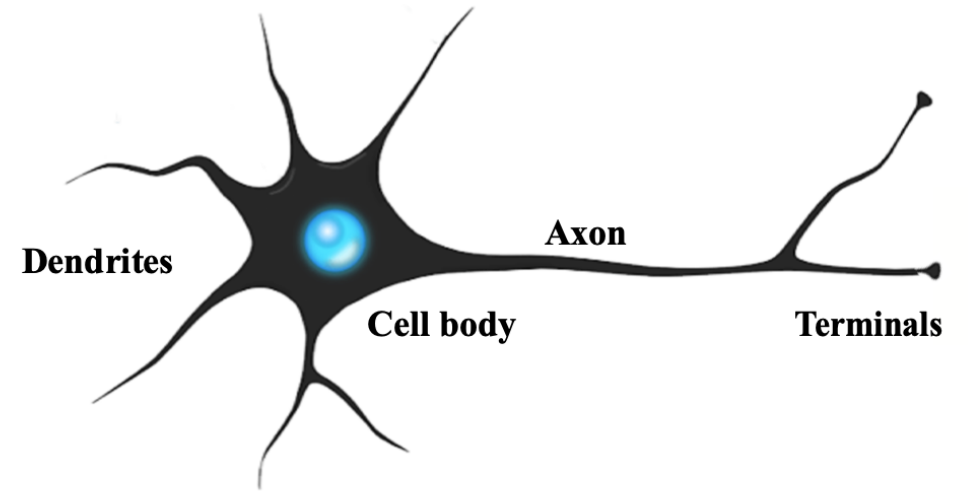


MODULE 1

INTRODUCTION TO NEURAL NETWORKS

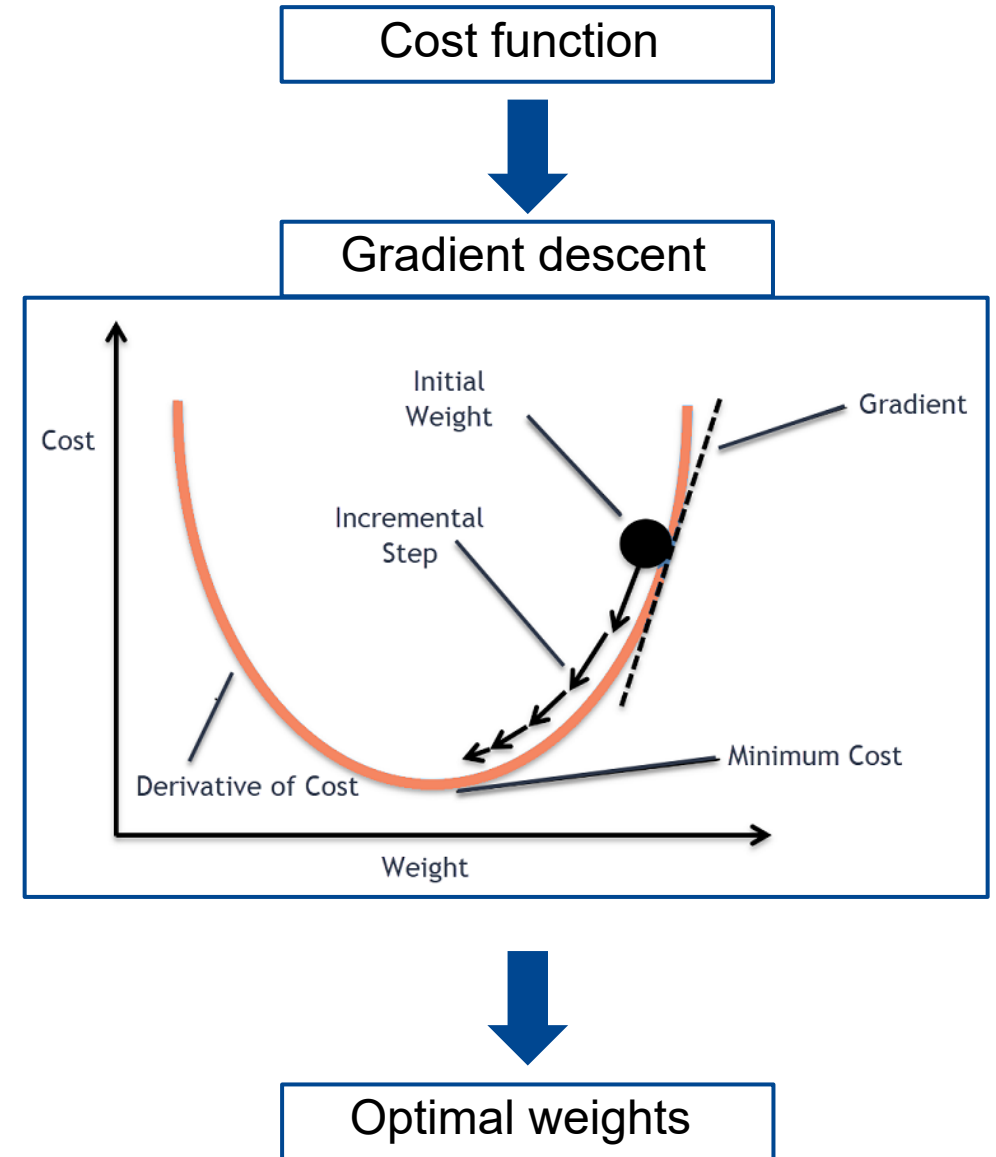
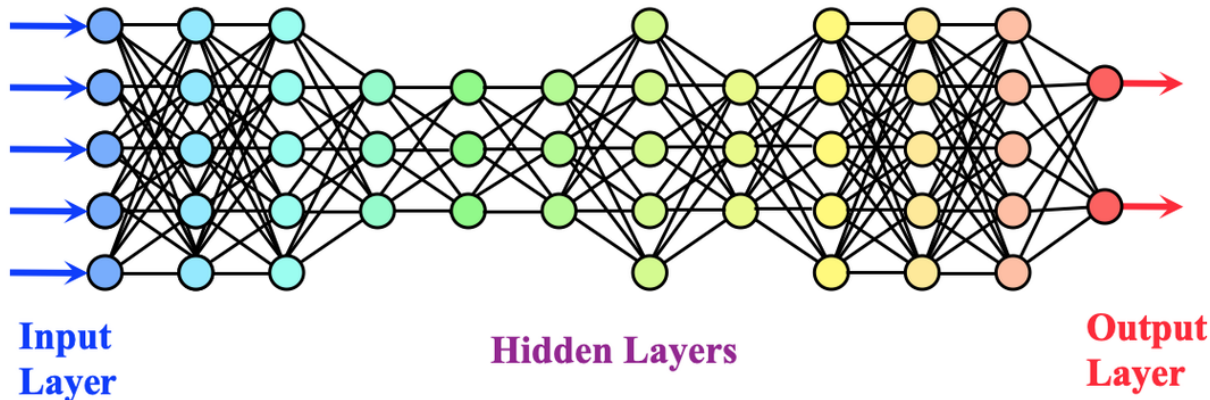
Neural Networks - Neurons

- **Neural networks** (NNs) are computational models inspired by the human brain's network of neurons.
- **Neurons** are the fundamental units of neural networks that process data.
- **Weights** determine the strength of the connection between neurons. They are adjusted during the training.
- **Activation functions** introduce non-linearity in the model.



Neural Networks - Training

- NNs are composed of several **layers** of neurons.
- NNs takes an input and yield an output (**functions**)
- The training is performed via **backpropagation algorithm** .
- **Cost function** = $(\text{Output layer} - \text{training data})^2$

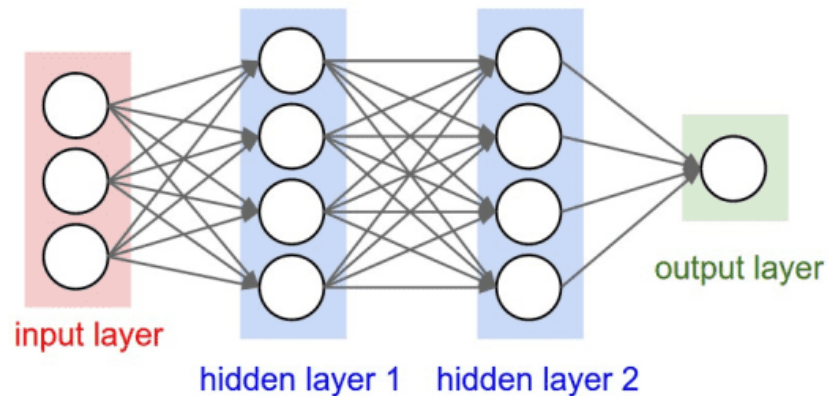


Neural Network Architectures (two examples)

- Different arrangements of neurons are called **architectures** .
- Different architectures are designed to be optimal for **different tasks** .

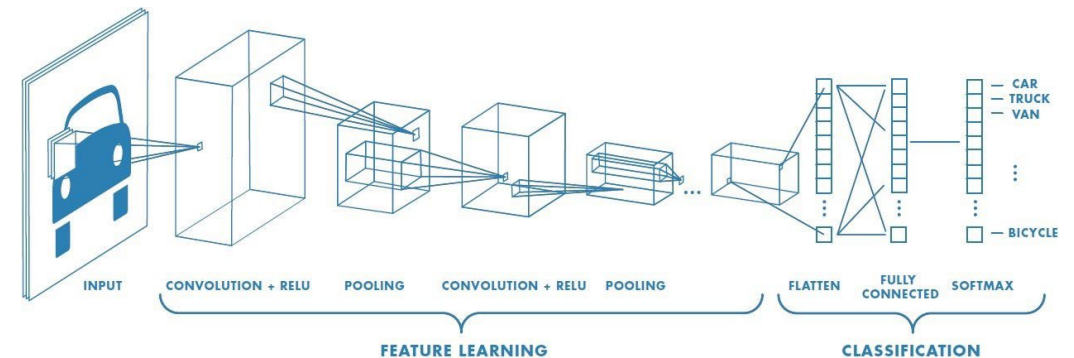
- **Feedforward Neural Network** :

- Fully-connected layers (“perceptron”)
- Used for classification and regression tasks
- Can approximate functions (!!!)



- **Convolutional Neural Network** :

- Convolutional layers (filters)
- Used for processing visual data
- Used in facial recognition and medical images



What are they good at?

- **Real-world applications :**
 - Identification of objects, facial recognition, medical diagnostics, autonomous vehicles.
- **Natural Language Processing :**
 - Understand human language, chatbots, language translation, sentiment analysis
- **Applications in architecture :**
 - Generative design, structural analysis, building information modelling (BIM)
- **Applications in acoustics :**
 - Acoustic feature recognition, shape optimization, and more on the second module...



What are they bad at?

- Neural networks often require **large amounts of high -quality data**
- NNs require significant **computational power** (bad for the environment and time -consuming)
- Overfitting: the model cannot generalise and **perform poorly on unseen data** .
- NNs are **difficult to interpret** and are usually seen as “black boxes”.

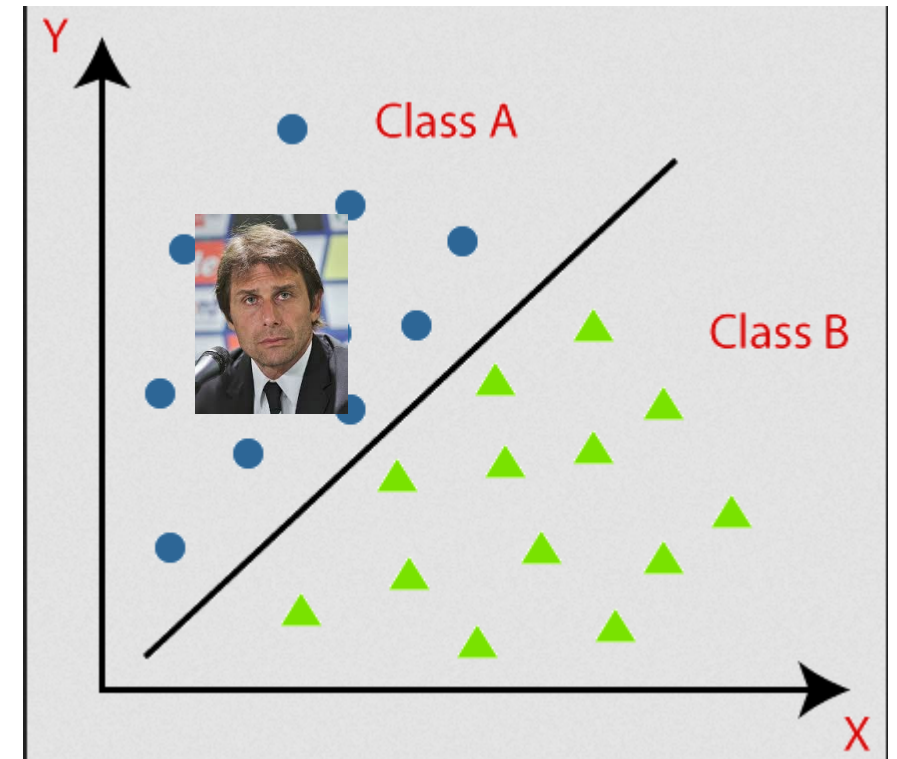


MODULE 2

INTRODUCTION TO REGRESSION PROBLEMS

What is a classification problem?

- In general, a classification problem involves assigning one or more categories (**classes**) to a given input based on its **features** .
- Examples: Emails (*spam* or *not spam*), Image (*cat* or *not cat*), Handwritten digit (which *number*), Antonio Conte* (*good coach* or *bad coach*).
- With **neural networks** , this is done by training a model to learn patterns in the data that correspond to different classes.
- Example: The number 8 has two “holes” -> the network learns this connection.

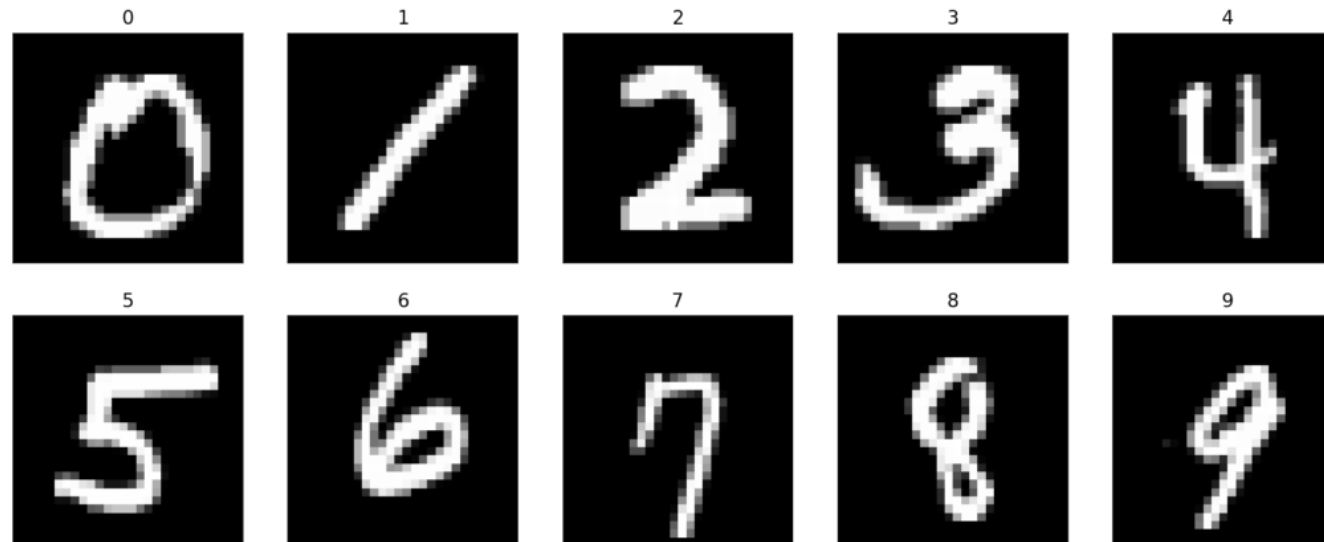


*He is clearly good.

Classify handwritten digits

We will consider as an example the following classification problem ([MNIST database](#)):

- Given an image of 28x28 pixels (**features**) representing a handwritten digit
- What number between 0 and 9 (**classes**) is represented?



- We will use a **Neural Network** to solve this problem!

1. Data pre-processing

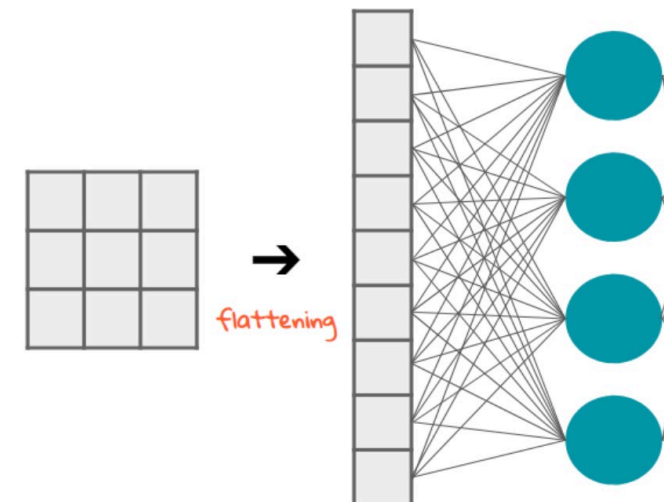
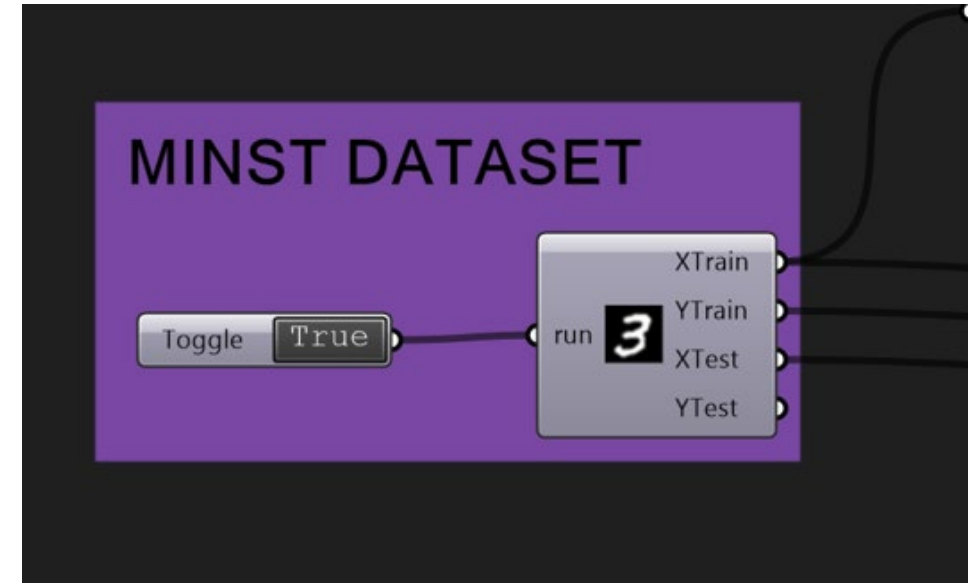
The first step is to **load the dataset**. Usually, this step come after the production of the dataset. In our case it is pre-made and can be downloaded directly.

```
# Load the MNIST dataset
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()
```

The second step is to flatten and normalize the images.

- **Flatten** an image: it transforms the image from a 2D arrangement of pixels into a long 1D vector.
- Our images are 28x28 pixels -> after flattening we have a long vector of 784 elements.
- **Normalize** an image: all the pixels have values between [0,1] (originally [0,255]).
- Automatically done in Grasshopper ☺

```
# Preprocess the data: Flatten images and normalize pixel values
x_train = x_train.reshape(-1, 28 * 28).astype('float32') / 255.0
x_test = x_test.reshape(-1, 28 * 28).astype('float32') / 255.0
```

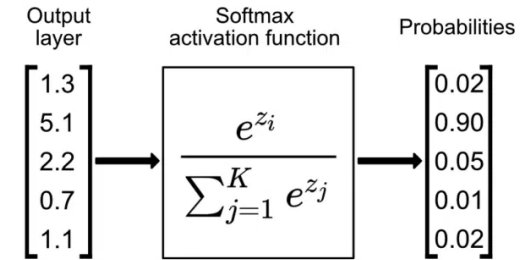


2. Define the neural network architecture

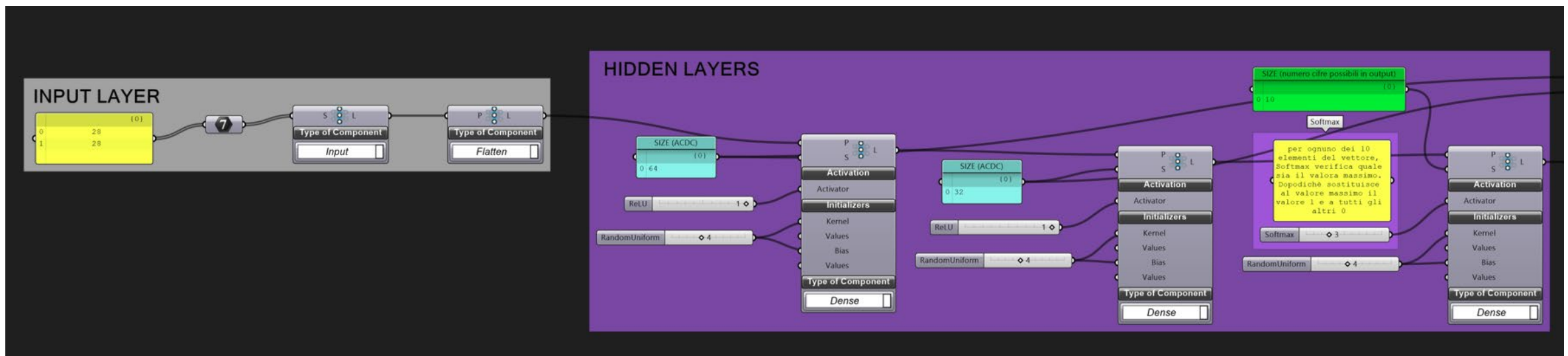
The next step is to build the **architecture of the neural network**.

The neural network has 4 layers:

- Input layer: 28x28 neurons (to receive the flatten images)
- Dense layer 1: 64 neurons
- Dense layer 2: 32 neurons
- Output layer: 10 neurons (10 classes for digits from 0 to 9)



```
# Define the FCNN model with an explicit Input layer
model = models.Sequential([
    layers.Input(shape=(28 * 28,)), # Explicit Input layer
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(10, activation='softmax') # 10 classes for digits 0-9
])
```

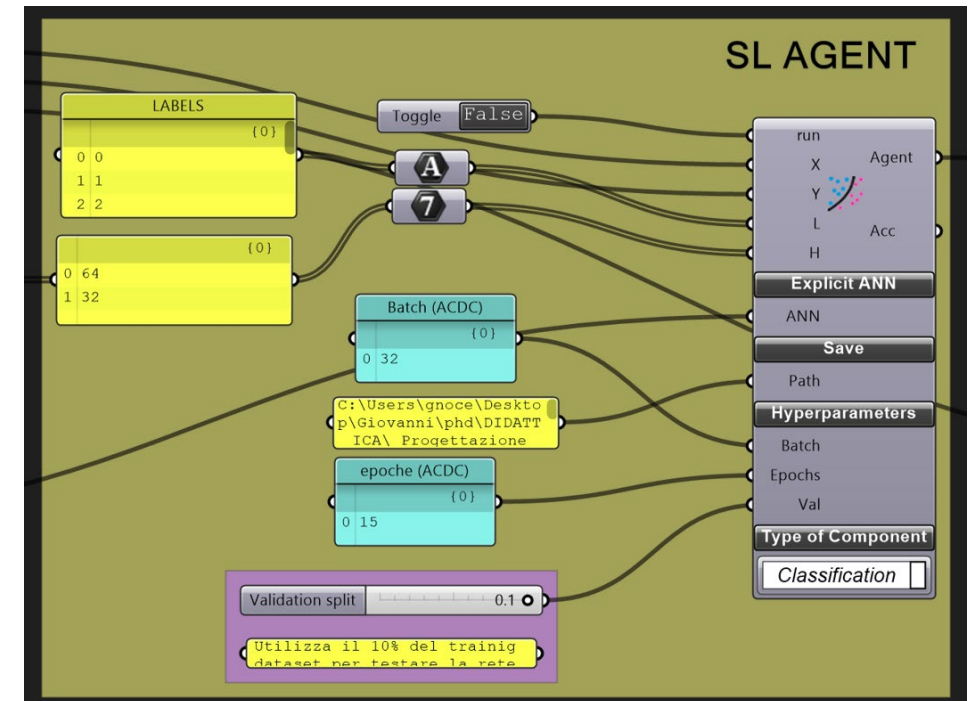
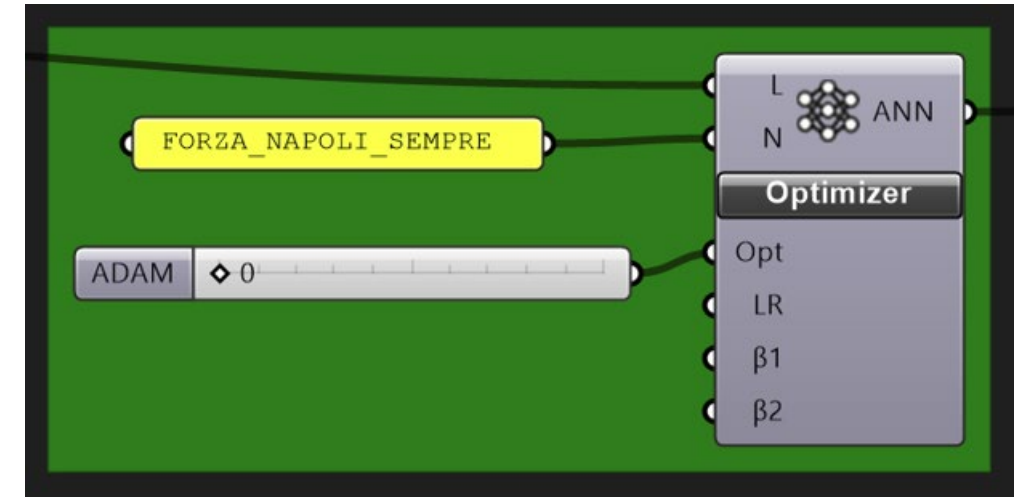


3. Compile the model

We need now to **compile the model**. This means that we must provide the following information:

- **Optimizer** : Adam (to solve the gradient descent)
- **Loss function** : Sparse categorical cross-entropy
 - Computes the difference between predicted and expected.
- **Metrics** : Accuracy
 - We check the accuracy of the results during training to take decisions (e.g., early stopping).

```
# Compile the model with optimizer, loss function, and evaluation metric
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```



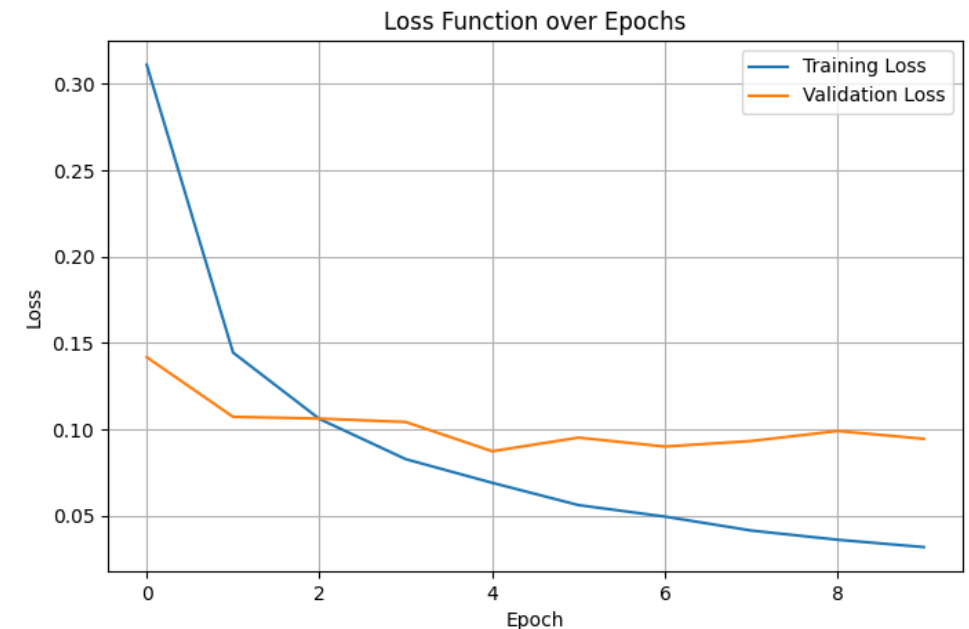
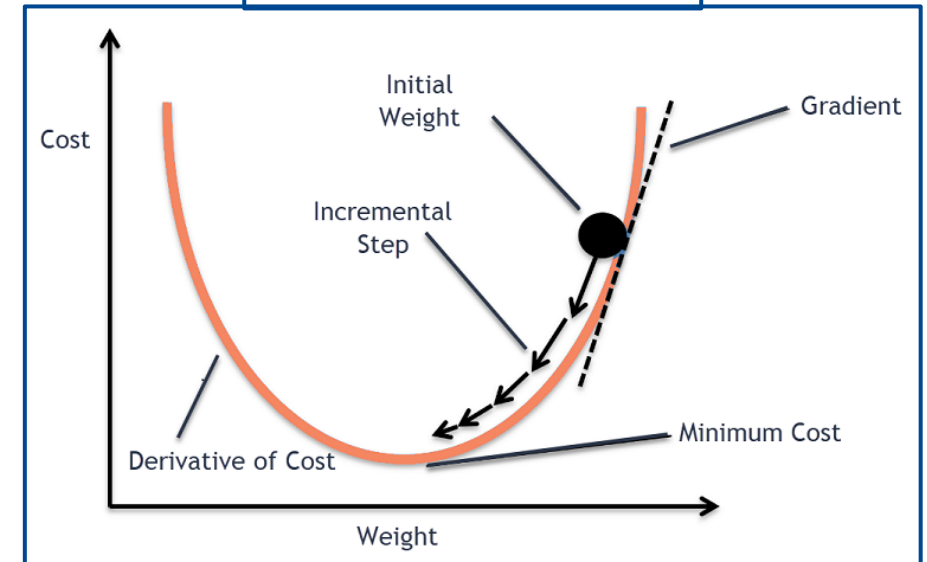
4. Train the neural network

The last step is to train the neural network .

- We need to provide the following information:
 - **Inputs** (from pre-processing)
 - **Epochs**: how many times we must process the dataset
 - **Batch size**: how many images we provide at the same time
 - **Validation** : 10% of images used for validation (**overfitting**)

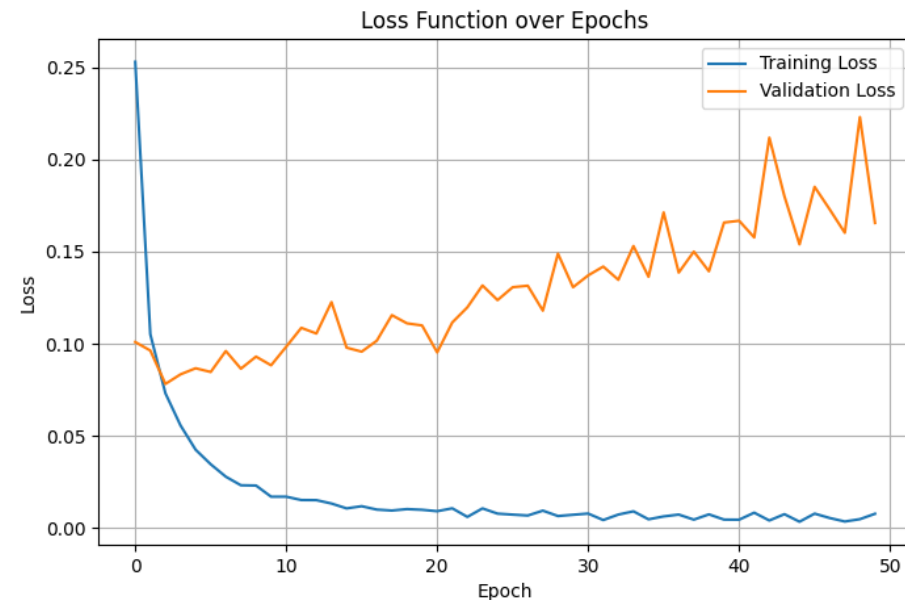
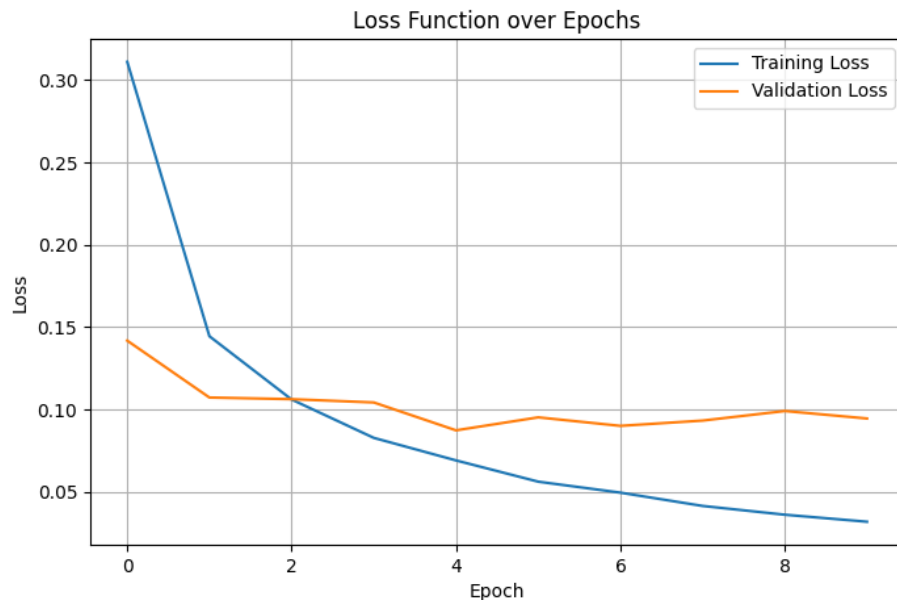
```
# Train the model using the training data
history = model.fit(
    x_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.1, # Use 10% of training data for validation
    verbose=2,
    callbacks=[checkpoint_callback, early_stopping]
)
```

Gradient descent



Be aware of overfitting

- **Overfitting** occurs when a neural network learns the details and noise in the training data to such an extent that it **performs poorly on new, unseen data**.
- Instead of generalizing to capture the underlying patterns in the data, **the model becomes overly specialized to the training dataset**, resulting in high accuracy on the training data but poor performance on validation or test datasets.
- Overfitting can be caused by **training for too long** (too many epochs!)



Good luck with your implementation!

- Later you will perform the previous steps in Grasshopper to implement a neural network able to recognize handwritten digits.
- Download this presentation from the repository and check the details of the implementation.

Thanks for your kind attention!

If you have additional questions, please send me an email at laudato@kth.se.

Ciao!

